설비 고장을 예측하는 AI 데이터 분석

Pandas 데이터 선택·필터링

표 합치기

posco

CONTENTS

목차

01 두 표를 키로 합치기 — merge 와 how

02 위아래로 이어 붙이기 — concat

03 종합 실습 — 합쳐서 라인별로 보기

posco

posco

01

표 합치기

키 열로 두 표를 이어 붙이기 — merge 와 how

표가 두 개면 분석이 막힌다

한 표에 없는 정보는 아무리 잘 걸러도 나오지 않는다

「A라인 불량률이 B라인보다 높은가」 —로그에는 라인 칸이 없다

조건 필터도 정렬도 이미 있는 칸에만 걸 수 있다. 로그의 칸 이름을 아무리 들여다봐도 라인이라는 칸 자체가 없으므로 이 질문은 시작조차 할 수 없다.

「도입 5년 넘은 설비가 더 나쁜가」 —로그에는 도입연도가 없다

도입연도는 설비 대장에만 있다. 설비ID는 두 표에 다 있으니 그것을 다리 삼아 도입연도를 로그 쪽으로 가져오면 그때부터 답할 수 있게 된다.

답에 필요한 칸이 다른 표에 있으면, 먼저 두 표를 합쳐야 한다

현장 데이터는 성격이 다른 정보를 따로 관리하기 때문에 거의 언제나 여러 표로 나뉘어 있다. 그래서 실무 분석은 합치기에서 시작하는 경우가 많다.

> SQL의 JOIN과 같은 일 —데이터분석 직무 면접에서 가장 자주 나오는 주제 중 하나

합치려면 두 표에 같은 칸이 있어야 한다

그 칸을 키 열이라 부른다 — 두 표를 잇는 다리

로그의 설비ID 와 대장의 설비ID — 이름도 값의 형식도 같다

키 열은 어려운 개념이 아니라 「두 표에 공통으로 있는 칸」이다. 여기서는 설비ID 이고, 양쪽 모두 DIE-A-01 형식으로 적혀 있다. 이름만 같고 형식이 다르면 소용이 없다.

로그의 DIE-A-01 을 대장에서 찾아, 그 줄의 라인·도입연도를 옆에 붙인다

하는 일은 단순한 반복이다. 로그 한 줄의 설비ID 를 읽고 → 대장에서 같은 값을 찾고 → 그 줄의 나머지 값을 옆에 붙인다. 이 일을 200줄에 대해 되풀이한다.

키가 될 수 있는 칸이 없으면 합칠 수 없다 — 먼저 만들어야 한다

앞 단원에서 설비ID 를 쪼개 라인을 만든 것이 그런 작업이다. 붙이기 전에 두 표의 columns 를 나란히 찍어 공통 칸을 눈으로 확인하는 것이 첫 습관이다.

> 이름이 같아도 값의 형식이 다르면 안 붙는다 — 뒤에 붙은 공백 한 칸으로도 짝을 못 찾는다

붙는 과정을 세 줄로 따라가기

200줄은 눈으로 못 보므로, 앞 세 줄만 떼어 확인한다

CODE

```
import pandas as pd
log = pd.read_csv('data/13_diecasting_shot.csv')
master = pd.read_csv('data/13_diecasting_master.csv')

three = log.head(3)[['샷', '설비ID', '비스킷두께']]   # 앞 세 줄 · 세 칸만
print(three.to_string(index=False))
#  샷      설비ID  비스킷두께
#  1  DIE-A-01   10.0
#  2  DIE-B-01   11.0

print(three.merge(master, on='설비ID').to_string(index=False))
#  샷      설비ID  비스킷두께 라인  도입연도  정격형체력 담당반
#  1  DIE-A-01   10.0  A   2018    280  1반
#  2  DIE-B-01   11.0  B   2020    320  2반
```

설비ID 는 그대로 두고 오른쪽에 라인·도입연도·정격형체력·담당반 네 칸이 붙었다 — 이것이 merge 의 전부다

merge 기본 사용법

왼쪽 표에 점을 찍고 merge, 괄호 안에 붙일 표와 키 열 이름

CODE

```
print(log.shape, master.shape)          # (200, 8) (5, 5)

merged = log.merge(master, on='설비ID')   # on = 두 표를 잇는 키 열
print(merged.shape)                     # (185, 12)

# 열은 8 + 5 - 1 = 12   키 열 설비ID 는 한 번만 남는다
print(merged[['샷', '설비ID', '라인', '도입연도']].head(2).to_string(index=False))
#  샷      설비ID 라인  도입연도
#  1  DIE-A-01   A   2018
#  2  DIE-B-01   B   2020
```

열 수는 8 +5 −1 =12 로 미리 계산되지만, 행 수는 붙여 봐야 안다 — 200이 아니라 185가 나왔다

posco

how — 어느 쪽을 다 남길 것인가

기본은 양쪽에 다 있는 줄만 남긴다 — 그래서 줄이 사라진다

how='inner' (기본) 양쪽 표에 모두 있는 키만 남김 — 200 → 185

대장에 없는 DIE-C-01 의 샷 15줄이 통째로 빠진다. how 를 안 적으면 자동으로 이 방식이고, 오류도 경고도 없이 조용히 빠진다는 점이 가장 위험하다.

how='left' 왼쪽 표는 다 남기고, 짝이 없으면 빈 칸 — 200 → 200

로그를 한 줄도 버리지 않는다. 짝을 못 찾은 15줄은 라인·도입연도가 빈 칸으로 남으므로 「어느 줄이 못 붙었는지」를 나중에 셀 수 있다.

how='outer' 양쪽을 다 남김 — 200 → 201

대장에만 있는 DIE-D-01 이 한 줄로 들어온다. 그 줄은 반대로 샷·압력이 전부 빈 칸이다. 「로그에 없는 설비」를 찾아내는 유일한 방법이다.

> 분석에서 기본값은 left — 원본 로그를 한 줄도 잃지 않고 정보만 더하는 쪽이 안전하다

posco

how를 바꿔 가며 행 수 비교

같은 두 표인데 how 하나로 185 • 200 • 201 이 된다

CODE

```
m_inner = log.merge(master, on='설비ID')                    # 기본 inner
m_left  = log.merge(master, on='설비ID', how='left')
m_outer = log.merge(master, on='설비ID', how='outer')
print(m_inner.shape, m_left.shape, m_outer.shape)
# (185, 12) (200, 12) (201, 12)

print(m_left['라인'].isna().sum())        # 15   짝을 못 찾아 빈 칸이 된 줄
print(m_left[m_left['라인'].isna()]['설비ID'].head(1).tolist())
# ['DIE-C-01']    대장에 없는 설비

print(m_outer[m_outer['샷'].isna()]['설비ID'].tolist())
# ['DIE-D-01']    로그에 없는 설비 — outer 로만 보인다
```

left 로 합친 뒤 붙여 온 열의 빈 칸을 세면 「짝을 못 찾은 줄」이 몇 줄인지 바로 나온다

posco

실습 1. 로그에 설비 정보 붙이기

두 파일을 읽어 공통 칸을 찾고, 그 칸을 기준으로 이어 붙이기

목표

로그에 없던 라인•도입연도를 대장에서 가져와 옆에 붙이기

단계

① 주조 로그 파일과 설비 대장 파일을 각각 읽어 크기를 출력

② 두 표의 칸 이름을 나란히 출력해 공통으로 있는 칸을 찾기

③ 그 칸을 기준으로 두 표를 합치고 크기를 출력

④ 합친 표의 앞 두 줄에서 라인 칸이 채워졌는지 눈으로 확인

예상 결과

로그 200줄 8칸 • 대장 5줄 5칸 • 합친 표는 185줄 12칸

실습 1 결과 해석

숫자 두 개 — 열은 예측되고 행은 예측되지 않는다

원리

키로 찾아 옆에 붙인다

로그의 한 줄에서 설비ID 를 읽고, 대장에서 같은 설비ID 를 찾아 그 줄의 나머지 값을 옆에 붙인다. 200줄에 대해 이 일을 반복한 것이다.

함정

행이 줄었는데 모르고 지나감

200줄이 185줄이 됐다. 짝을 못 찾은 15줄이 조용히 사라진 것이다. 합치기 전후 행 수를 안 찍으면 알아챌 방법이 없다.

검산

8 + 5 − 1 = 12

열은 두 표의 칸 수를 더하고 키 열 하나를 뺀 값이다. 12가 아니면 키 열 이름을 잘못 적었거나 공통 칸이 둘 이상인 것이다.

posco

실습 1 정답

읽고 •공통 칸을 찾고 •합치는 세 단계

CODE

```
import pandas as pd

log = pd.read_csv('data/13_diecasting_shot.csv')
master = pd.read_csv('data/13_diecasting_master.csv')
print(log.shape, master.shape)       # (200, 8) (5, 5)

print(list(log.columns))
# ['샷', '설비ID', '실린더압력', '주조압력', '사이클타임', '비스킷두께', '형체력', '품질등급']
print(list(master.columns))
# ['설비ID', '라인', '도입연도', '정격형체력', '담당반']   → 공통은 설비ID

merged = log.merge(master, on='설비ID')
print(merged.shape)                  # (185, 12)   행이 15줄 줄었다
print(merged[['샷', '설비ID', '라인', '도입연도']].head(2).to_string(index=False))
#  샷      설비ID 라인  도입연도
#  1  DIE-A-01   A   2018
```

출처 — KAMP 제조AI 데이터셋 「주조 설비 예지보전」(중기부·KAIST) 발췌 · 설비 대장은 교육용 구성

합치면 행이 줄기도 늘기도 한다

줄면 짝을 못 찾은 것, 늘면 오른쪽 표에 같은 키가 여러 줄인 것

줄었다 → 왼쪽에만 있는 키가 있다

how='left' 로 바꿔 합친 뒤 붙여 온 열의 빈 칸을 세면 어느 줄인지 바로 나온다. 앞에서 배운 방법 그대로다.

그대로다 → 모든 키가 짝을 찾았다

가장 바람직한 상태다. 다만 오른쪽 표에 키 중복이 없다는 조건이 함께 맞아야 행 수가 유지된다는 점을 잊지 말 것.

늘었다 → 오른쪽 표에 같은 키가 두 줄 이상 있다

가장 자주 놓치는 경우다. 대장에 같은 설비가 두 번 등록되어 있으면 왼쪽 한 줄이 두 줄로 불어나고, 합계나 평균이 그 설비만 두 배로 반영된다.

> 합친 뒤 행 수가 원본과 다르면 분석을 멈추고 원인부터 찾는다 —데이터가 아니라 진단의 문제

행이 늘어나는 경우를 직접 만들어 보기

오른쪽 표에 키가 두 줄이면 왼쪽 한 줄이 두 줄이 된다

CODE

```
# 대장에 DIE-A-01 이 실수로 두 번 등록됐다고 가정
master_dup = master.iloc[[0, 0, 1, 2, 3, 4]]   # 앞 단원 iloc 그대로
print(len(master_dup), master_dup['설비ID'].nunique())
# 6 5      ← 줄 수 6 인데 종류는 5 · 한 설비가 두 번 들어 있다

m_dup = log.merge(master_dup, on='설비ID')
print(m_dup.shape)                    # (248, 12)   185 가 아니라 248

print(len(log[log['설비ID'] == 'DIE-A-01']))   # 63   이 줄들이 두 배가 됐다
# 185 + 63 = 248
```

len 과 nunique 가 다르면 그것만으로 경고 신호 — 붙이기 전에 오른쪽 표에서 확인할 것

posco

합치기 전후 점검 코드

세 줄이면 된다 — 원본 행 수 · 키 중복 · 합친 뒤 행 수

CODE

```
print(len(log))                                  # 200   합치기 전
print(len(master), master['설비ID'].nunique())   # 5 5   대장에 키 중복 없음

m_left = log.merge(master, on='설비ID', how='left')
print(len(m_left))                               # 200   한 줄도 잃지 않음
print(m_left['라인'].isna().sum())               # 15    정보를 못 붙인 줄

# 어느 설비가 문제인지까지
unmatched = m_left[m_left['라인'].isna()]
print(unmatched['설비ID'].head(1).tolist(), len(unmatched))
# ['DIE-C-01'] 15
```

nunique 는 값의 종류 수 — 대장 5줄에 설비 5종이면 키가 겹치지 않는다는 뜻

posco

실습 2. 합친 뒤 행 수로 검산하기

how를 바꿔 가며 행 수를 비교하고, 사라진 줄이 어느 설비인지 찾기

목표

세 가지 how 의 행 수를 비교하고 원인 설비를 찾아내기

단계

① 로그와 대장을 읽어 기본 방식으로 합친 뒤 줄 수를 적기

② 왼쪽 표를 다 남기는 방식으로 다시 합쳐 줄 수를 적기

③ 두 결과의 줄 수 차이를 계산하고, 그만큼이 어디로 갔는지 말해 보기

④ 왼쪽을 다 남긴 표에서 라인 칸이 빈 줄만 골라 설비 이름과 줄 수를 확인

예상 결과

기본 185줄 • 왼쪽 전부 200줄 • 차이 15줄 • 빈 칸인 줄은 전부 DIE-C-01

실습 2 결과 해석

세 숫자와 그 차이가 말해 주는 것

원리

how 는 짝 없는 줄을 어떻게 할지 정한다

inner 는 버리고, left 는 남기되 빈 칸으로 두고, outer 는 양쪽 다 남긴다. 표의 내용은 그대로인데 how 하나로 결과가 달라진다.

함정

185를 그냥 받아들임

기본값이 inner 라서 아무 말 없이 15줄이 사라진다. 오류도 경고도 없다. 합치기 전 행 수를 모르면 사라진 사실 자체를 알 수 없다.

검산

200 − 185 = 15

차이 15가 left 결과의 빈 칸 15와 정확히 같아야 한다. 두 숫자가 다르면 대장 쪽에 키 중복이 있다는 신호다.

사라진 15줄은 DIE-C-01 — 데이터가 아니라 대장 갱신이 늦은 것이 원인이다

실습 2 정답

how를 바꿔 가며 행 수를 비교하고 원인을 찾는다

CODE

```
log = pd.read_csv('data/13_diecasting_shot.csv')
master = pd.read_csv('data/13_diecasting_master.csv')

m_inner = log.merge(master, on='설비ID')                  # 기본 inner
m_left  = log.merge(master, on='설비ID', how='left')
print(len(m_inner), len(m_left))       # 185 200
print(len(m_left) - len(m_inner))      # 15    사라진 줄 수

unmatched = m_left[m_left['라인'].isna()]   # 앞 단원 조건 필터 그대로
print(len(unmatched))                  # 15
print(unmatched['설비ID'].head(1).tolist())   # ['DIE-C-01']

m_outer = log.merge(master, on='설비ID', how='outer')
print(len(m_outer))                    # 201   대장에만 있는 DIE-D-01 이 더해짐
```

차이 15와 빈 칸 15가 같다 — 두 숫자가 어긋나면 대장 쪽 키 중복을 의심할 것

posco

02

이어 붙이기

칸이 같은 표를 위아래로 쌓는 concat

posco

pd.concat — 위아래로 이어 붙이기

merge 는 옆으로 칸을 붙이고, concat 은 아래로 줄을 쌓는다

CODE

```
# 로그를 두 번에 나눠 받았다 가정 (각각 0번부터 번호가 매겨져 있음)
part1 = log.iloc[:100].reset_index(drop=True)
part2 = log.iloc[100:].reset_index(drop=True)
print(part1.shape, part2.shape)        # (100, 8) (100, 8)

stacked = pd.concat([part1, part2])    # 대괄호로 묶어 넘긴다
print(stacked.shape)                   # (200, 8)   칸 수는 그대로
print(stacked.index[-3:].tolist())     # [97, 98, 99]   번호가 겹친다
print(len(stacked.loc[[0]]))           # 2   0번이 두 줄
```

형태 주의 — merge 는 표.merge(다른 표), concat 은 pd.concat([표1, 표2]) 다

posco

ignore_index — 번호를 다시 매기기

붙이면서 0부터 새로 매기거나, 붙인 뒤 다시 매기거나

CODE

```
renumbered = pd.concat([part1, part2], ignore_index=True)
print(renumbered.index[-1])            # 199   0부터 다시 매겨졌다
print(len(renumbered.loc[[0]]))        # 1     0번이 한 줄

# 앞 단원 도구로 나중에 매겨도 결과는 같다
same_result = pd.concat([part1, part2]).reset_index(drop=True)
print(same_result.index[-1])           # 199
print(same_result.equals(renumbered))  # True   두 길이 같은 표를 만든다
```

붙이면서 매기든 붙인 뒤 매기든 결과는 같다 — 어느 쪽이든 번호를 정리하고 넘어갈 것

posco

칸이 다른 표를 붙이면 — 조용히 생기는 빈 칸

오류가 아니라 빈 칸으로 채워진다 — 그래서 더 위험하다

CODE

```
# 2차 파일에 형체력 칸이 빠져 있다고 가정
keep = ['샷', '설비ID', '실린더압력', '주조압력',
        '사이클타임', '비스킷두께', '품질등급']
part2_short = part2[keep]              # 앞 단원 복수 열 선택
print(part2_short.shape)               # (100, 7)   형체력이 없다

missing_col = pd.concat([part1, part2_short], ignore_index=True)
print(missing_col.shape)               # (200, 8)   오류 없이 붙는다
print(missing_col['형체력'].isna().sum())   # 106
print(part1['형체력'].isna().sum())     # 6     원래 있던 결측
# 6 + 100 = 106   뒤쪽 100줄이 통째로 빈 칸이 됐다
```

붙이기 전에 두 표의 columns 를 비교할 것 — 칸 이름 뒤의 공백 한 칸만으로도 다른 칸이 된다

posco

merge 와 concat — 언제 무엇을

붙이는 방향이 다르고, 필요한 조건도 다르다

merge 옆으로 칸을 붙임 · 키 열이 있어야 함 · 행 수가 달라질 수 있음

「정보를 더한다」면 merge 다. 로그에 라인·도입연도를 붙이는 일이 여기 해당한다. 붙인 뒤에는 행 수가 원본과 같은지를 본다.

concat 아래로 줄을 쌓음 · 칸 이름만 맞으면 됨 · 행 수는 반드시 더해짐

「같은 표를 모은다」면 concat 이다. 1월 로그와 2월 로그를 모으는 일이다. 붙인 뒤에는 칸 수가 늘지 않았는지, 빈 칸이 갑자기 생기지 않았는지를 본다.

판단 기준은 하나 — 무엇을 더하려는가

칸을 더하려면 merge, 줄을 더하려면 concat. 이 한 문장으로 거의 모든 경우가 정리된다. SQL 로는 merge 가 JOIN, concat 이 UNION ALL 이다.

> 둘 다 붙인 뒤에는 shape 를 찍는다 — merge 는 행 수를, concat 은 칸 수와 빈 칸을 본다

posco

실무에서는 둘을 이어서 쓴다

월별 파일을 모아 한 해 로그를 만들고, 거기에 대장을 붙인다

CODE

```
# ① 같은 표를 모은다 — concat
year = pd.concat([part1, part2], ignore_index=True)
print(year.shape)                          # (200, 8)   칸 수 그대로, 줄만 더해짐

# ② 정보를 더한다 — merge
report = year.merge(master, on='설비ID', how='left')
print(report.shape)                        # (200, 12)  줄 수 그대로, 칸만 더해짐
print(report['라인'].isna().sum())          # 15   붙지 않은 줄은 여기서 드러난다

# concat 뒤에는 칸 수를, merge 뒤에는 줄 수를 본다
```

이 두 줄이 실무 분석의 시작 — 파일을 모으고 기준정보를 붙인 다음에야 집계가 시작된다

posco

03

종합 실습

합치고 · 거르고 · 세워 라인별 점검표 만들기

posco

실습 3. 합쳐서 라인별로 보기

합치기와 앞 두 단원의 조건·정렬을 한 줄기로 이어 점검표 만들기

목표

라인 정보를 붙인 뒤 양품이 아닌 샷을 위험한 순서로 정리

단계

① 로그와 대장을 왼쪽 표가 다 남도록 합치기

② 라인이 비어 있는 줄을 걸러내고 남은 줄 수를 확인

③ 남은 표에서 품질등급이 양품이 아닌 줄만 고르기

④ 비스킷두께가 큰 순서로 세워 샷·설비·라인·비스킷두께·품질등급만 상위 5줄 출력

예상 결과

라인이 붙은 줄 185 • 양품 아닌 줄 34 • 맨 위는 샷 198 • A라인 • 27.0 • 불량

실습 3 결과 해석

오늘 배운 한 줄이 앞 두 단원과 어떻게 이어지는가

원리

합치고 · 거르고 · 세우는 순서

먼저 필요한 칸을 붙이고, 그다음 조건으로 행을 줄이고, 마지막에 줄을 세운다. 붙이기를 나중에 하면 걸러낸 표에 다시 붙여야 해서 손이 두 번 간다.

함정

라인 빈 칸을 안 걸러냄

DIE-C-01 15줄은 라인이 빈 칸이다. 이 줄들을 그냥 두고 라인별로 세면 어느 라인에도 안 들어가면서 전체 합만 안 맞는다.

검산

185 · 34 · 샷 198

라인이 붙은 줄 185, 그중 양품이 아닌 줄 34, 맨 위가 샷 198 이면 맞다. 185 는 앞 실습의 inner 결과와 같은 숫자다.

실습 3 정답

합치고 • 거르고 • 세우는 네 줄

CODE

```
log = pd.read_csv('data/13_diecasting_shot.csv')
master = pd.read_csv('data/13_diecasting_master.csv')

joined = log.merge(master, on='설비ID', how='left')   # ① 붙이기
with_line = joined[joined['라인'].notna()]            # ② 라인 없는 줄 제외
print(len(with_line))                                  # 185

not_ok = with_line[with_line['품질등급'] != '양품']    # ③ 양품 아닌 줄만
print(len(not_ok))                                     # 34

result = not_ok.sort_values('비스킷두께', ascending=False)   # ④ 두꺼운 순
print(result[['샷', '설비ID', '라인', '비스킷두께', '품질등급']]
      .head(5).to_string(index=False))
#  샷      설비ID 라인  비스킷두께 품질등급
# 198  DIE-A-01   A    27.0   불량
# 192  DIE-B-01   B    26.0   불량
```

새로 배운 줄은 ① 하나 — ②③④ 는 앞 두 단원에서 이미 한 것